PRCV PROJECT (COMPUTER VISION BASED)

Real-Time Hand Sign Detection
Lightweight Landmark-Based Recognition from a Webcam

Aim:

Real-time detection of hand signs from a laptop webcam using MediaPipe hand landmarks and a small neural network trained on a public dataset.

Objective:

To recognise five common signs (Hello, I Love You, Peace, Thank You and Yes) on a MacBook without any cloud API, to show the tracked hand and the confidence of each sign live, and to measure accuracy on people the model was not trained on.

Theory:

Dataset: Sign_Language_Dataset has 1,770 webcam-style images from about three people (A, B and C), in six gesture folders. Its "ok" folder shows two raised fingers rather than the usual OK gesture, so those images were used as "no sign" examples. SignAlphaSet supplied further "no sign" examples: 40 images each of 18 alphabet letters, leaving out letters that overlap the five signs.

Models Compared:

1. Logistic Regression on x, y, z landmarks
2. Neural network with one hidden layer of 32 units on x, y, z landmarks
3. Neural network with hidden layers of 64 and 32 units on x, y, z landmarks
4. Neural network with hidden layers of 64 and 32 units on x, y landmarks (selected)

- MediaPipe Hand Landmarker is a pretrained model that finds a hand and returns 21 keypoints.
- Normalisation moves the wrist to the origin and divides by the hand's size, so the features do not depend on where the hand is or how far it is from the camera.
- The selected network maps 42 inputs to six outputs (five signs and "no sign"). It has 5,030 parameters and takes 23 KB on disk.
- Temperature scaling (T = 1.9) softens overconfident outputs. A sign is named only when its smoothed confidence reaches 60%.

Evaluation Metrics: Macro-F1, per-sign recall, false-accept rate, confusion matrix, processing time per frame, CPU use and memory. Testing is leave-one-person-out: the images are burst photos, so a random split would put near-identical frames in both training and test sets and inflate the score.

Methodology

1. Install MediaPipe, OpenCV, NumPy and scikit-learn in a virtual environment.
2. Download both datasets and verify their published checksums.
3. Extract hand landmarks from each image (hand found in 1,766 of 1,770 gesture images).
4. Group images by person, using clothing colour clusters as an approximate identity.
5. Normalise the landmarks and build the feature vectors.

6. Augment training data with mirroring, rotation up to 20° and small landmark noise, and balance the classes by resampling.
7. Compare the four models with leave-one-person-out testing over five random seeds.
8. Set the temperature and confidence threshold from the held-out results.
9. Train the selected model on all 2,476 usable images and export its weights for NumPy.
10. Check the final model on images of the letter V, which it never trained on.
11. Build the real-time application with OpenCV and test it on video files.

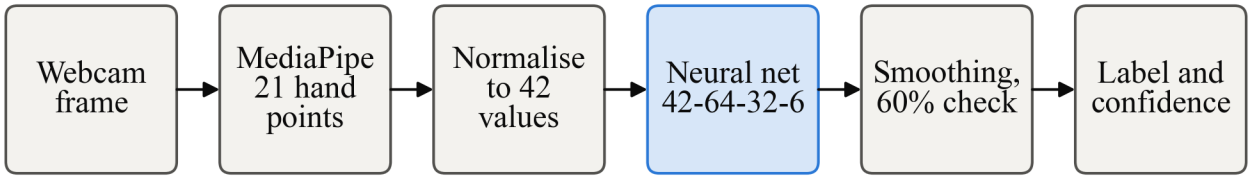


Fig. 1: Real-time pipeline. Only the neural net (blue) was trained in this project.



Fig. 2: The five signs and a "no sign" example, with landmarks. Images cropped and annotated from the two datasets (CC BY 4.0).

Real-Time Application Interface

The application is an OpenCV desktop window that runs entirely on the laptop. It acts as the presentation layer between the webcam and the trained model. The interface allows users to:

- See the tracked hand skeleton and bounding box on a mirrored camera view.
- Read the recognised sign and its confidence next to the hand.
- Compare live confidence bars for all five signs and "no sign".
- Build a line of text from signs held for 0.6 seconds, and clear it with a key press.
- Monitor the frame rate and the processing time per frame.

Output:

Table 1: Model comparison on held-out people: macro-F1 and accuracy per person, averaged over five seeds, before the 60% threshold.

Model	Macro-F1	Person A	Person B	Person C
Logistic Regression	0.910 ± 0.007	99.0%	71.1%	98.4%
Net (32), x, y, z	0.902 ± 0.023	98.2%	66.0%	99.6%
Net (64, 32), x, y, z	0.907 ± 0.037	97.9%	68.3%	99.6%
Net (64, 32), x, y	**0.958 ± 0.006**	**100%**	**81.6%**	**100%**

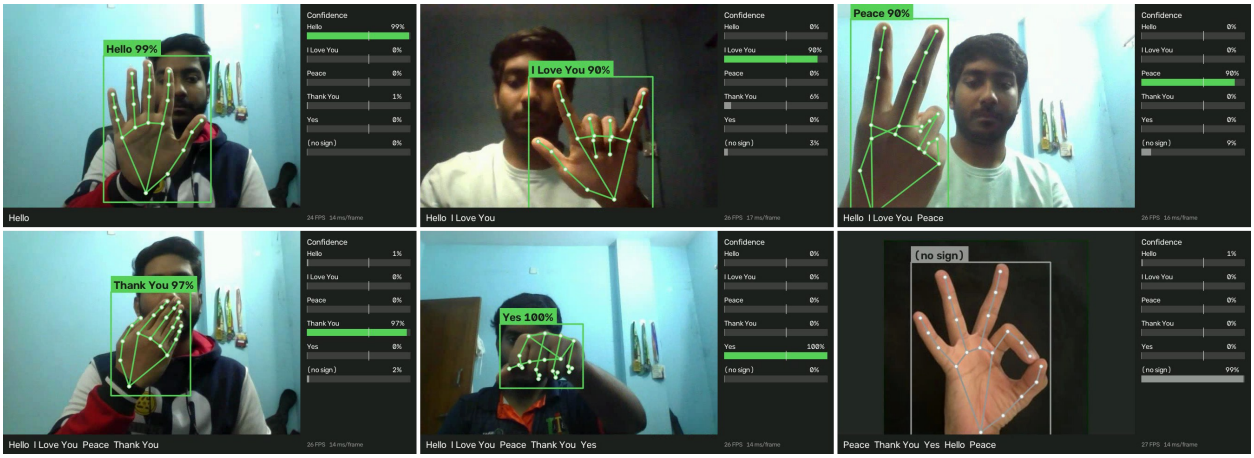


Fig. 3: Application output on a test video built from dataset images, some of which were used in training. This shows the interface working, not its accuracy.

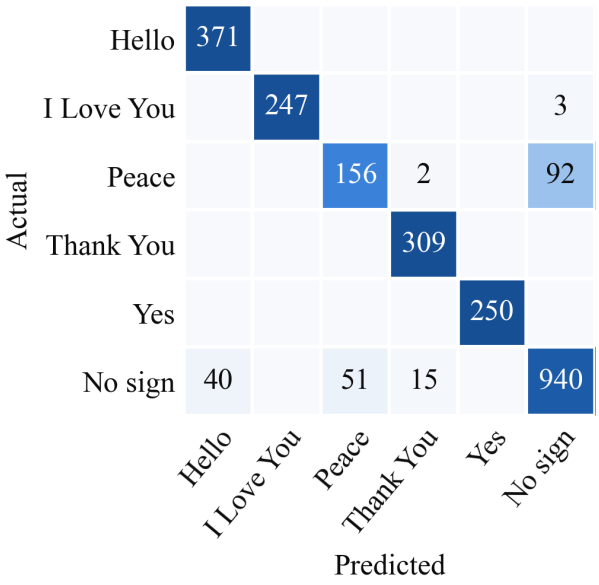


Fig. 4: Held-out confusion matrix at the 60% threshold (one seed).

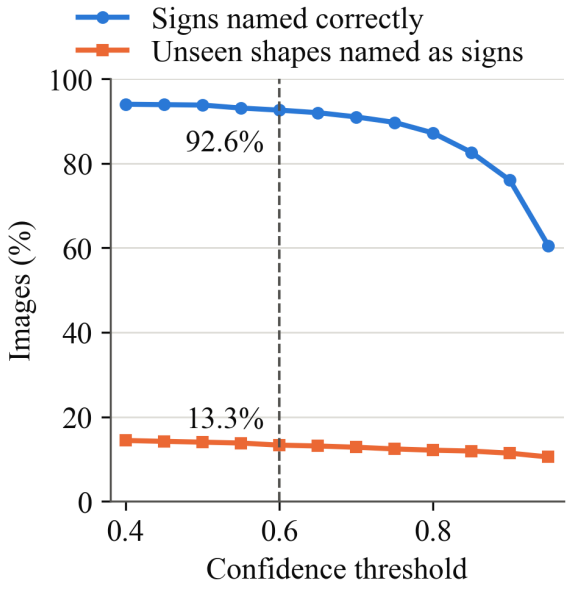


Fig. 5: Effect of the threshold (mean of five seeds). The dashed line marks the 60% used.

Results: At the 60% threshold, 92.6% (± 1.0) of held-out signs were named correctly. Recall in Fig. 4 was 100% for Hello, Thank You and Yes, 98.8% for I Love You and 62.4% for Peace. Peace is partly understated: with Person B held out, only 33 Peace images remained for training. On the outside check, the letter V, which has the same handshape as Peace, was named Peace in 80% of 40 images. Fig. 5 shows that raising the threshold rejects few unfamiliar shapes (13.3% were still named as signs at 60%) while rejecting more real signs.

Table 2: Runtime on an M2 MacBook Air (8 GB RAM); per-frame figures from test video.

Measure	Result
Processing time per frame	14 ms median, 26 ms at 95th percentile
CPU and peak memory	About 50% of one core; about 220 MB
Classifier size and prediction time	23 KB; about 11 microseconds
Landmark extraction; evaluation and training	70 s; 30 s

Limitations:

- The system recognises static hand poses, not full sign language. Real Hello, Thank You and Yes use movement, and signing also uses the face, body and grammar.
- Testing covers about three people, unevenly (Fig. 6): Person A has no I Love You or Peace images. The model and threshold were also chosen on these held-out results, so expect lower accuracy on new hands.
- Testing so far used video files. A live webcam run is still pending.



Fig. 6: Images used per sign and person, grouped approximately by clothing colour.

Learning Outcomes:

- Learned hand landmark extraction using MediaPipe.
- Designed position- and scale-invariant features from keypoints.
- Trained and compared lightweight classifiers for real-time use.
- Learned why leave-one-person-out testing is needed to avoid data leakage.
- Applied confidence calibration and open-set rejection through a “no sign” class.
- Built a real-time computer vision application using OpenCV.

References:

1. Sign_Language_Dataset, Mendeley Data, CC BY 4.0. doi.org/10.17632/pwyrdzwzwf.1
2. SignAlphaSet, Mendeley Data, CC BY 4.0. doi.org/10.17632/8fmvr9m98w.3
3. Google MediaPipe Hand Landmarker. ai.google.dev/edge/mediapipe